CAL POLY POMONA

MAT 3800

Operation Research

Linear Programming for Optimizing Transportation Cost and Time

Authors
Jeremy Lin
Thuy Lu

Academic Mentor
Dr. Jillian Cannons

December 10, 2020

Contents

Introduction **2**

Base Problem **2**
Problem Description 2
Mathematical Formulation 2
Results and Discussion 4

Extension Problem **5**
Problem Description 5
Mathematical Formulation 5
Results and Discussion 6

Conclusion **9**

Appendix **9**
Linear Programming Code 9

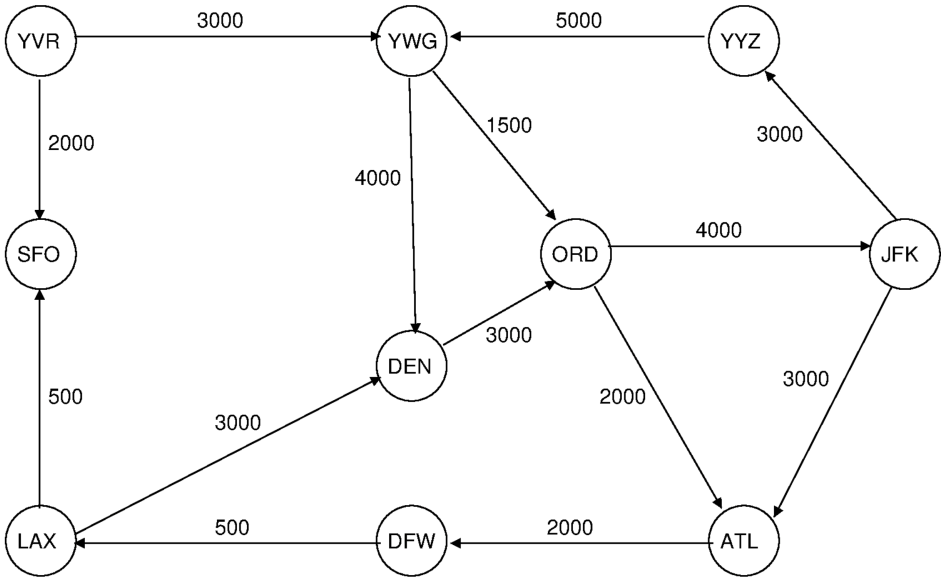

Figure 1: Flight routes and capacities

Introduction

In this project, we are considering a distribution problem of a major North American technology company that manufacture their products oversea and ship them to ports in North America. The company needs to determine how to transport the products to other distribution centers across Canada and the United States. We are considering a base and an extension problem where the company could have a flexibility in choosing whether to transport their products as air freight or a combination of air and ground freight and uses Linear Programming to minimize the cost and/or time of transportation.

Base Problem

Problem Description

In the base problem, we formulate a linear programming model to allocate numbers of units transported to different distribution centers with the objective of minimizing the air freight cost. The cost to transport one unit of product is \$5.79 / kilometer. The 10 distribution centers together with their units received and demanded are given in Table 2. Unfortunately, air freight options are only available between certain cities and routes are subject to various capacities, as described in the given Figure 1. From the figure, we have a total of 14 routes, each denoted by an index i. The destinations of the routes and the total flight distances between each of the cities can be found in Table 1.

Mathematical Formulation

With the goal of minimizing the air freight cost, we formulate the objective function as the product of the cost to transport one unit of product (\$5.79) and the sum of all number of units transported on route i multiplied by its corresponding route distance. Denote d_i c_i as the air distance and capacity of route i, respectively, and x_i as the number of units

of product transported on route i for $i = 1, 2, ..., 14$. We place constraints on the flow at each distribution center such that the difference between the number of units demanded / shipped out and the number of units received from oversea / other distribution centers equals 0. In addition, as the routes have certain capacities, we place constraints on each x_i that they have to be less than or equal to their capacity c_i. Subsequently, we have the linear programming problem with 14 decision variables x_i:

$$\text{Minimize } Z = 5.79 \sum_{i=1}^{14} d_i x_i$$

subject to

$$\begin{aligned}
x_1 + x_2 &= 3325 && (1)\\
x_3 + x_4 - x_1 - x_5 &= -750 && (2)\\
x_5 - x_8 &= -2930 && (3)\\
-x_2 - x_{11} &= -885 && (4)\\
x_6 + x_7 - x_3 - x_{10} &= -2715 && (5)\\
x_8 + x_9 - x_6 &= 4400 && (6)\\
x_{10} - x_4 - x_{12} &= -620 && (7)\\
x_{11} + x_{12} - x_{13} &= 2000 && (8)\\
x_{13} - x_{14} &= -1340 && (9)\\
x_{14} - x_9 - x_7 &= -485 && (10)\\
x_i &\leq c_i && (11)
\end{aligned}$$

and $x_i \geq 0$ for $i = 1, 2, ..., 14$

Route			**Length of route (km)**		**Capacity of route (unit)**	
Index i	**From**	**To**	**By air**	**By ground**	**By air**	**By ground**
1.	Vancouver(YVR)	Winnipeg (YWG)	1865	2303	3000	4500
2.	Vancouver(YVR)	San Francisco (SFO)	1270	1538	2000	3000
3.	Winnipeg(YWG)	Chicago (ORD)	1130	1371	1500	2250
4.	Winnipeg (YWG)	Denver (DEN)	1285	1762	4000	6000
5.	Toronto (YYZ)	Winnipeg (YWR)	1500	2031	5000	7500
6.	Chicago (ORD)	New York (JFK)	1120	1348	4000	6000
7.	Chicago (ORD)	Atlanta (ATL)	965	1198	2000	3000
8.	New York (JFK)	Toronto (YYZ)	590	792	3000	4500
9.	New York (JFK)	Atlanta (ATL)	1310	1440	3000	4500
10.	Denver (DEN)	Chicago (ORD)	1455	1597	3000	4500
11.	Los Angeles (LAX)	San Francisco (SFO)	560	623	500	750
12.	Los Angeles (LAX)	Denver (DEN)	1335	1700	3000	4500
13.	Dallas (DFW)	Los Angeles (LAX)	1970	2325	500	750
14.	Atlanta (ATL)	Dallas (DFW)	1180	1299	2000	3000

Table 1: Length and capacity of air / ground routes

Distribution Center	Units Demanded	Units Received
Vancouver(YVR)	675	4000
Winnipeg(YWG)	750	-
Toronto (YYZ)	2930	-
San Francisco (SFO)	885	-
Chicago (ORD)	2715	-
New York (JFK)	8625	13025
Denver (DEN)	620	-
Los Angeles (LAX)	4000	6000
Dallas (DFW)	1340	-
Atlanta (ATL)	485	-

Table 2: Number of units demanded and received at each distribution center

Results and Discussion

We implement our model in `MATLAB`. For our linear programming approach, we construct the adjacency matrix and use it to formulate the constraints and objective function. We then use the function `linprog` from `MATLAB` to obtain the optimal solutions. As a result, the minimum cost of shipping is found to be $104,215,513. And, the optimal number of units to be transported on each of the 14 routes available is listed in Table 3.

Route			Optimal solution
Index i	**From**	**To**	**Number of units transported**
1.	Vancouver(YVR)	Winnipeg (YWG)	2440
2.	Vancouver(YVR)	San Francisco (SFO)	885
3.	Winnipeg(YWG)	Chicago (ORD)	1500
4.	Winnipeg (YWG)	Denver (DEN)	190
5.	Toronto (YYZ)	Winnipeg (YWR)	0
6.	Chicago (ORD)	New York (JFK)	0
7.	Chicago (ORD)	Atlanta (ATL)	355
8.	New York (JFK)	Toronto (YYZ)	2930
9.	New York (JFK)	Atlanta (ATL)	1470
10.	Denver (DEN)	Chicago (ORD)	1570
11.	Los Angeles (LAX)	San Francisco (SFO)	0
12.	Los Angeles (LAX)	Denver (DEN)	2000
13.	Dallas (DFW)	Los Angeles (LAX)	0
14.	Atlanta (ATL)	Dallas (DFW)	1340
Minimum transportation cost			**$104,215,513**

Table 3: Optimal solution to the base problem

Extension Problem

Problem Description

Air freight is known to be the best solution when it comes to transportation due to its highly reliable schedule and large network of destinations. However, at times it might be costly. Thus, the company considers using ground transportation in addition to air freight. The cost to transport one unit as ground freight is \$2.79 / kilometer. To avoid model complexity, we assume the ground routes are the same as the air routes described in the graph in the given Figure 1. Since ground freight allows larger units to be transported, we modify the routes' capacities to be 1.5 time as much as their prior values and the capacities are shown in Table 1. Though ground freight costs less than air freight, it takes longer to transport a shipment. We need to take the speed of the transporting vehicle into consideration, here we assume the average speed of air and ground travelling is 835 km/h and 79 km/h respectively. As a result, we modify the objective function to be a linear combination of the transportation cost and the transportation time such that the company could obtain minimized values for both. Realizing that the company might want to change its priority in minimizing cost or time, we find optimal values for the objective function with different linear combinations to provide the company with the flexibility of choosing their focus on minimizing transportation cost or time.

Mathematical Formulation

Our objective shifts from minimizing cost in the base problem to minimizing both costs and time, so introduce two new variables namely *cost* and *time* to generate the objective function that now is a sum of a weight w_1 times *cost* and weight w_2 times *time*. The weights' function is to give different selections of preferred percentage of focus on minimizing cost or time. Thus, the sum of w_1 and w_2 is 1, hence one weight's value is the difference between 1 and the other. Now that we include ground transportation with the same routes as air freight, our constraints are modified accordingly. In addition to our existing variables, denote d'_i c'_i as the ground distance and capacity of route i, respectively, and x'_i as the number of units of product transported on route i for $i = 1, 2, ..., 14$. The constrained of flow at the distribution centers are adjusted to include units transported in and out by ground freight as well. Also, the ground routes have different capacities than air ones (1.5 times as much), we added constraints where x'_i has to be less than or equal to c'_i.

With *cost* and *time*, we added one constraint for each of them. Aiming for minimizing the two values, we want the difference between the variables and what they are defined by to be equal to 0. Defining *cost* is similar to how the objective function was defined in the base problem, but now include both costs by air and ground transportation. Moreover, we scale these values by a factor of 10^{-6} in order to create a comparable value tom time variable when minimizing our objective function. We calculate time from each route by dividing the distance of each route by the speed of the vehicle used. Since our variable is the number of units of product, we scale the variable x_i by dividing it by the capacity of the route to ensure that our time is in hour. By dividing it with this scaling factor, we assume that the time needed to transport products from each distribution center is proportional to the number of items shipped, i.e. the fewer number of items ship from distribution center. the faster the time spent on transporting the items.

We have the following linear programming problem:

$$\text{Minimize } Z = (w_1 \times cost) \ + \ (w_2 \times time)$$

subject to

$$\begin{aligned}
x_1 + x_1' + x_2 + x_2' &= 3325 && (1)\\
x_3 + x_3' + x_4 + x_4' - x_1 - x_1' - x_5 - x_5' &= -750 && (2)\\
x_5 + x_5' - x_8 - x_8' &= -2930 && (3)\\
-x_2 - x_2' - x_{11} - x_{11}' &= -885 && (4)\\
x_6 + x_6' + x_7 + x_7' - x_3 - x_3' - x_{10} - x_{10}' &= -2715 && (5)\\
x_8 + x_8' + x_9 + x_9' - x_6 - x_6' &= 4400 && (6)\\
x_{10} + x_{10}' - x_4 - x_4' - x_{12} - x_{12}' &= -620 && (7)\\
x_{11} + x_{11}' + x_{12} + x_{12}' - x_{13} - x_{13}' &= 2000 && (8)\\
x_{13} + x_{13}' - x_{14} - x_{14}' &= -1340 && (9)\\
x_{14} + x_{14}' - x_9 - x_9' - x_7 - x_7' &= -485 && (10)\\
cost - \frac{5.79}{10^6}\sum_{i=1}^{14} d_i x_i - \frac{2.76}{10^6}\sum_{i=1}^{14} d_i' x_i' &= 0 && (11)\\
time - \frac{1}{835}\sum_{i=1}^{14} \frac{d_i}{c_i} x_i - \frac{1}{79}\sum_{i=1}^{14} \frac{d_i'}{c_i'} x_i' &= 0 && (12)\\
x_i &\le c_i && (13)\\
x_i' &\le c_i' && (14)
\end{aligned}$$

$$\text{and } x_i \ge 0, \ x_i' \ge 0 \text{ for } i = 1, 2, ..., 14$$

Results and Discussion

Similar to the base problem, we implement our new model in `MATLAB` to obtain optimal solution. In order to analyze the different optimal solutions due to the different weights, starting from 0, we increment w_1 by 0.001 and plot the resulting optimum transportation time and cost for all the weight values. Figure 2 shows that when w_1 is small, our model places priority in minimizing the time spent on transportation, thus results in lower time, but relatively higher cost (w_1 =0.01, w_2 =0.99). Similarly when w_1 is large, our model makes minimizing cost a priority, and thus results in lower transportation cost but longer time (w_1 =0.99, w_2 =0.01).

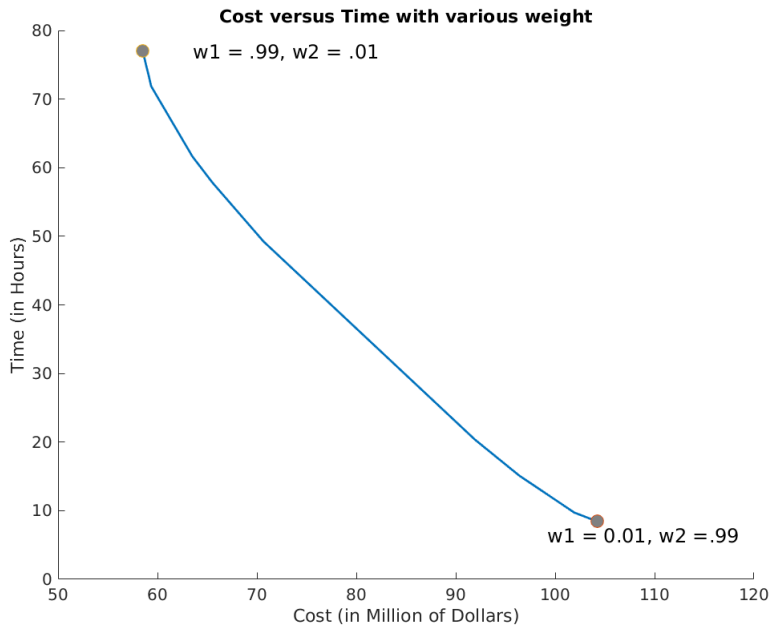

Figure 2: Transportation cost compared to Time for shipping

Table 4 shows 4 different representation of how *cost* and *time* related with different weight function. As the weight of *cost* increases (w_1) we can observe that the total shipping cost decreases and the total *time* increase. This means that our linear programming will use ground shipping more than air shipping since ground shipping is cheaper, but relatively slower than air shipping.

w_1, w_2 / Objectives	0.1, 0.99	0.4, 0.6	0.7, 0.4	0.9, 0.1
Cost *(million of dollar)*	104.2	101.9	63.5	58.5
time *(hour)*	8.4	9.7	61.6	77.0

Table 4: values of objective functions at different weights

Figure 3 that shows optimum unit distribution from each 14 different possible routes among the distribution center for these 4 different variation of weights. The light grey bar represents flight shipment from each distribution center while dark grey bar represents the number of units shipped using ground shipment. As the value of w_1 increases, which minimizing cost are more important than minimizing time, it is more optimal to ship our product using ground shipment as it is cheaper, although it requires longer time,

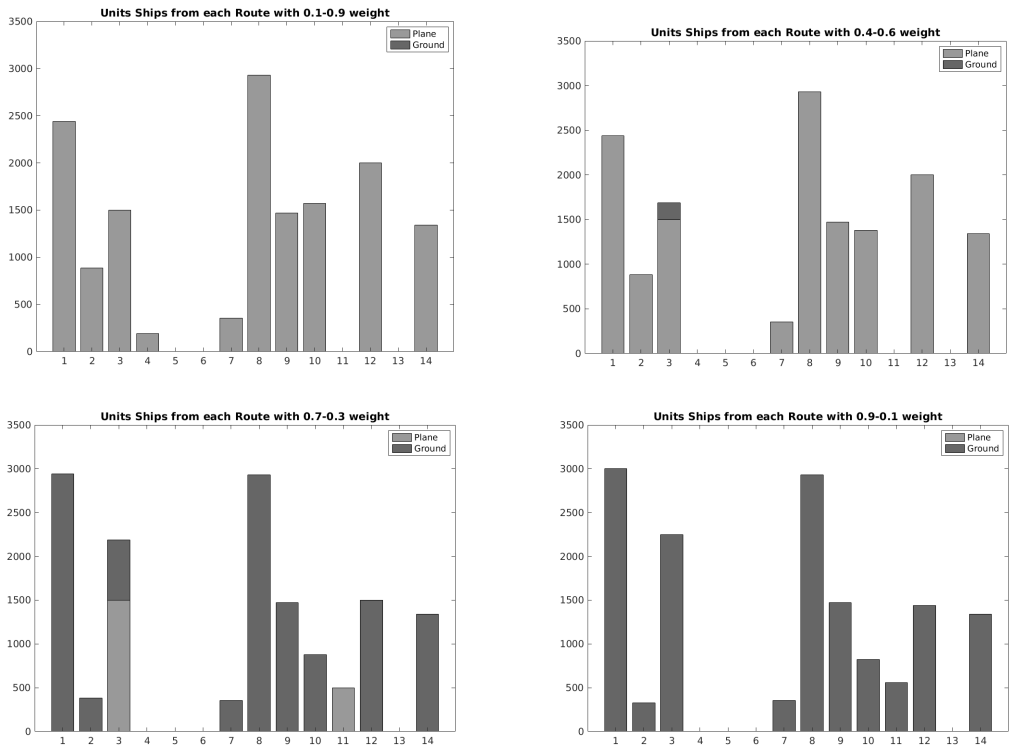

Figure 3: Units transported from each route

To visualize how weights affect the value of our overall objective function, we plotted the objective function with respect to the changes of weight values. The two lines represent the change of Z with respect to w_1 and w_2, which is a mirror image of each other. As weight 1 increases, the overall Z values increases until around 0.7 where the overall objective function starts to decrease again, which indicates a possible local maximum which need to be investigated further.

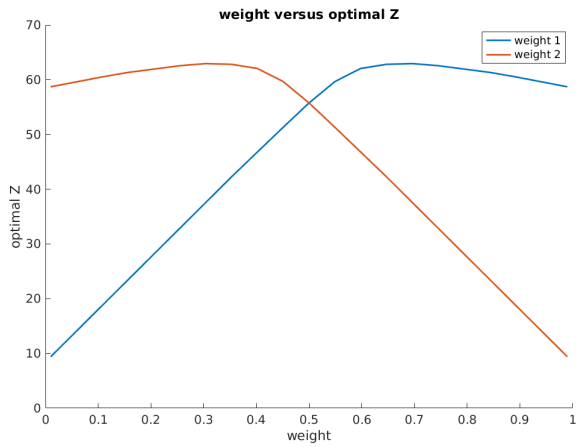

Figure 4: Value of objective function plotted with different weight function

Conclusion

We worked on finding the most optimal way to ship products to distribution centers via specific routes. Each distribution center requires different number of units, constrained to the capacity of units that can be shipped through each shipping route. We considered two different models to explore the possible optimal solution to this distribution problem. Our first model in the base problem, we are trying to minimize transportation cost on the assumption that we can only ship out items via air freight. We formulated our model and find the optimized solution through `linprog()` command in `MATLAB` . We then worked on an extensionof the base problem where we allow a cheaper, but slower transporting option through ground freight. We formulated our linear programming problem to minimize both cost and time spent on shipping. We introduce weight variable as a tuning parameter to determine which of the two objective function to prioritize to obtain different optimal solutions. The scope for our future work includes using Mixed Integer Linear Programming to better model the time variable constraint in our model. A further investigation of a possible local maximum on weight could be conducted to see whether there exists an *efficient frontier* due to the opposite effect of minimizing time and cost.

Appendix

Linear Programming Code

```
function[optZ,optDV,fPlaneCargo,fTruckCargo,fCost,fTime]=Project_3800_ver2
(w1,w2)
%This is script to solve Final Project of LP
%================List of Variable=======================================
%-----------------------Output Variable-----------------------------------
%optZ optimal Objective function
%optDV optimal number of unit transported for each route
%fPlaneCargo number of units shipped using air shipping
    %from all distribution
%fTruckCargonumber of units shipped using ground shipping
    %from all distribution
%fCost Total cost of
%fTime Total time in objective function
%---------------Special Variable for distribution problem-----------------
% acName is array that stores the name of all Distribution Centers
% aafRoute is adjacency matrix that stores route capacity information from
%           origin node(row) to destination node(col)
% aafXDVReverseLookupDV is matrix that stores the information of each route
%                        from and where the route of each XDV

%----------------- General Variable for LP --------------------------------
% nDV is how many Decission variables we have (number of col of A matrix)
% nConstraintLEQ is how many less or equal constraint we have (# of Row
% aafALEQ is the less or equal constraint matrix
% afbRHSLEQ is the array that saves the right hand side of less or eaual to
% aafAEQ is th equality constraints
```

```
% afBRHSEQ is the right hand side of equality constraint
% nEQCounter counter for equality constraints
% optDV  optimal weights of cargo 1,2,3,4 assign to which part of the plane
% optZ optimal Z profit of the plane
% afc is coeffiecient of Z
% afLB is the lowerbound array of DV's
% afUB is the upperbound array of DV'snd


%%%
```

```
%%%
```

```
    PlaneConstraintsLEQ(aafACoefLEQ,afbRHSLEQ,...
    aafPlaneRoute,aafPlaneRouteLookup,aafPlaneRouteReverseLookup,...
    aafTruckRoute,aafTruckRouteLookup,aafTruckRouteReverseLookup,...
    afUnitSupply,afUnitDemand,nDistCenter,cConstraintsLEQ,...
    nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV);

%==========================================================================

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Get rid of unused constraint memory
aafACoefEQ = aafACoefEQ(1:cConstraintsEQ,:);
afbRHSEQ =afbRHSEQ(1:cConstraintsEQ);
aafACoefLEQ = aafACoefLEQ(1:cConstraintsLEQ,:);
afbRHSLEQ =afbRHSLEQ(1:cConstraintsLEQ);


%Create LB
afLB = zeros(1,nDV);
%Create UB
afUB = [];

%Create Objective function
[C]=CreateObjective(aafFlightDistance,aafPlaneRoute,fPlaneCost,...
    aafTruckDistance,aafTruckRoute,fTruckCost,...
    aafPlaneRouteLookup,aafPlaneRouteReverseLookup,...
    aafTruckRouteLookup,aafTruckRouteReverseLookup,...
    nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV,...
    fPlaneSpeed,fTruckSpeed,w1,w2);

%Find the most optimized Solution
[optDV,optZ] = linprog(C,aafACoefLEQ,afbRHSLEQ,....
                        aafACoefEQ,afbRHSEQ,afLB,afUB);

 fPlaneCargo = optDV(1:nXPlaneDV);
 fTruckCargo = optDV(nXPlaneDV+1:nXTruckDV);
 fCost = optDV(nCostDV);
 fTime = optDV(nTimeDV);
%==========================================================================
% PrintSolution(nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV,...
%                          aafPlaneRouteReverseLookup,...
%                          aafTruckRouteReverseLookup,...
%                          acName,optDV,optZ)
end

function[]=PrintSolution(nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV,...
                        aafPlaneRouteReverseLookup,...
                        aafTruckRouteReverseLookup,...
```

```
                        acName,optDV,optZ)

     fTotalCost = optDV(nCostDV);
     fTotalTime = optDV(nTimeDV);
     fprintf('The minimum  shipping cost')
     fprintf('is %.3f in ten of million dollars \n',fTotalCost)
     fprintf('The minimum time shipping time is ')
     fprintf('%.3f in hours \n',fTotalTime)
    for i = 1:nXPlaneDV
        PrintPlaneRoute(i,optDV(i),aafPlaneRouteReverseLookup,...
               acName)
    end

    for i = nXPlaneDV+1:nXTruckDV
        PrintTruckRoute(i-nXPlaneDV,optDV(i),aafTruckRouteReverseLookup,...
               acName)
    end
end

function[C]=CreateObjective(aafFlightDistance,aafPlaneRoute,fPlaneCost,...
    aafTruckDistance,aafTruckRoute,fTruckCost,...
    aafPlaneRouteLookup,aafPlaneRouteReverseLookup,...
    aafTruckRouteLookup,aafTruckRouteReverseLookup,...
    nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV,...
    fPlaneSpeed,fTruckSpeed,w1,w2)
%cost is in terms of million of dollars
C = zeros(1,nDV);
C(nCostDV)=w1;
C(nTimeDV)=w2;
end

function[aafACoefEQ,afbRHSEQ,cConstraintsEQ]=...
    ConstraintsEQ(aafACoefEQ,afbRHSEQ,...
    aafPlaneRouteLookup,aafPlaneRouteReverseLookup,...
    aafTruckRouteLookup,aafTruckRouteReverseLookup,...
    aafFlightDistance,aafPlaneRoute,fPlaneCost,...
    aafTruckDistance,aafTruckRoute,fTruckCost,...
    afUnitSupply,afUnitDemand,nDistCenter,cConstraintsEQ,...
    nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV,fPlaneSpeed,fTruckSpeed)

for iConstraint = 1:nDistCenter
    cConstraintsEQ = cConstraintsEQ+1;
    NewConstraint = zeros(1,nDV);
    %Adding The Out Flow
        for iOut = 1:nDistCenter
            if aafPlaneRoute(iConstraint,iOut)>0
                NewConstraint(aafPlaneRouteLookup(iConstraint,iOut)) = 1;
            end
```

```
            if aafTruckRoute(iConstraint,iOut)>0
                NewConstraint(aafTruckRouteLookup(iConstraint,iOut)) = 1;
            end
        end

     %Adding In-flow
        for iIn = 1:nDistCenter
            if aafPlaneRoute(iIn,iConstraint)>0
                NewConstraint(aafPlaneRouteLookup(iIn,iConstraint)) = -1;
            end

            if aafTruckRoute(iIn,iConstraint)>0
                NewConstraint(aafTruckRouteLookup(iIn,iConstraint)) = -1;
            end
        end
    aafACoefEQ(iConstraint,:) = NewConstraint;

    % Right hand side of the equality constraint (supply-demand)
    afbRHSEQ (iConstraint) = ...
        afUnitSupply(iConstraint)-afUnitDemand(iConstraint);
end

%Adding Constraint for Cost
cConstraintsEQ = cConstraintsEQ+1;
NewConstraint = zeros(1,nDV);
for i = 1:nXPlaneDV
    %dummy cariable store direction of flow
    dummy = aafPlaneRouteReverseLookup(i,:);
    NewConstraint(i) = ...
        -10^-6*fPlaneCost*aafFlightDistance(dummy(1),dummy(2));
end
for i = nXPlaneDV+1:nXTruckDV
    %dummy cariable store direction of flow
    dummy = aafTruckRouteReverseLookup(i-nXPlaneDV,:);
    NewConstraint(i) = ...
        -10^-6*fTruckCost*aafTruckDistance(dummy(1),dummy(2));
end
NewConstraint(nCostDV) = 1;
aafACoefEQ(cConstraintsEQ,:) = NewConstraint;

%Adding Time Constraint
cConstraintsEQ = cConstraintsEQ+1;
NewConstraint = zeros(1,nDV);
for i = 1:nXPlaneDV
    %dummy cariable store direction of flow
    dummy = aafPlaneRouteReverseLookup(i,:);
    %fprintf('variable%d ...
    %have constraint %.3f\n',i,aafPlaneRoute(dummy(1),dummy(2)))
```

```
    NewConstraint(i) = ...
        -1*(1/fPlaneSpeed)*(1/aafPlaneRoute(dummy(1),dummy(2)))...
        *aafFlightDistance(dummy(1),dummy(2));
end
for i = nXPlaneDV+1:nXTruckDV
    %dummy cariable store direction of flow
    dummy = aafTruckRouteReverseLookup(i-nXPlaneDV,:);
%fprintf('variable%d have constraint...
%%.3f\n',i,aafTruckRoute(dummy(1),dummy(2)))
    NewConstraint(i) = -1*(1/fTruckSpeed)*...
        (1/aafTruckRoute(dummy(1),dummy(2)))*...
        aafTruckDistance(dummy(1),dummy(2));
end
NewConstraint(nTimeDV) = 1;
aafACoefEQ(cConstraintsEQ,:) = NewConstraint;

end

function[aafACoefLEQ,afbRHSLEQ,cConstraintsLEQ]=...
    PlaneConstraintsLEQ(aafACoefLEQ,afbRHSLEQ,...
    aafPlaneRoute,aafPlaneRouteLookup,aafPlaneRouteReverseLookup,...
    aafTruckRoute,aafTruckRouteLookup,aafTruckRouteReverseLookup,...
    afUnitSupply,afUnitDemand,nDistCenter,cConstraintsLEQ,...
    nDV,nXPlaneDV,nXTruckDV,nCostDV,nTimeDV)

% This Function build Inequality Constraint for LP problem

 for iConstraint = 1:nXPlaneDV
            cConstraintsLEQ = cConstraintsLEQ+1;
            NewConstraint = zeros(1,nDV);
            NewConstraint(iConstraint)=1;

         % Filling in LHS less equal constraint
            aafACoefLEQ(cConstraintsLEQ,:) = NewConstraint;

         % Filling in RHS less equal than constrint
            dummy = aafPlaneRouteReverseLookup(iConstraint,:);
            afbRHSLEQ (cConstraintsLEQ) = ...
                aafPlaneRoute(dummy(1),dummy(2));
 end

for iConstraint = nXPlaneDV+1:nXTruckDV
            cConstraintsLEQ = cConstraintsLEQ+1;
            NewConstraint = zeros(1,nDV);
            NewConstraint(iConstraint)=1;

         % Filling in LHS less equal constraint
            aafACoefLEQ(cConstraintsLEQ,:) = NewConstraint;
```

```
        % Filling in RHS less equal than constraint
           dummy = aafTruckRouteReverseLookup(iConstraint-nXPlaneDV,:);
           afbRHSLEQ (cConstraintsLEQ) = ...
               aafTruckRoute(dummy(1),dummy(2));
end
end

function[]=PrintPlaneRoute(nPlaneVariable,fAns,...
    aafPlaneRouteReverseLookup,...
    acName,optDV)
%This is a print function that tells which variable represent the answer
%for plane route
%nPlaneVAriable represent the XDV number (1-14)
fprintf("Plane cargo from %s to %s carry %d items \n",...
    acName(aafPlaneRouteReverseLookup(nPlaneVariable,1)),...
    acName(aafPlaneRouteReverseLookup(nPlaneVariable,2)),...
    fAns)

end

function[]=PrintTruckRoute(nTruckVariable,fAns,...
    aafTruckRouteReverseLookup,...
    acName,optDV)
%This is a print function that tells which variable represent the answer
%for plane route
%nPlaneVAriable represent the XDV number (1-14)
fprintf("Truck cargo from %s to %s carry %d items \n",...
    acName(aafTruckRouteReverseLookup(nTruckVariable,1)),...
    acName(aafTruckRouteReverseLookup(nTruckVariable,2)),...
    fAns)

end

function[aafTruckRouteLookup,aafTruckRouteReverseLookup,cDV]=...
    buildXDVTruckLookup(aafTruckRoute,cDV)
nSize = size(aafTruckRoute,1);
%Pre-allocate Variable
    % Plane look up table
    aafTruckRouteLookup = zeros(nSize);
    % Plane reverse lookup
    aafTruckRouteReverseLookup = zeros(nSize*nSize,2);

nCount=0;
for irow = 1:nSize
    for icol = 1:nSize
       if aafTruckRoute(irow,icol) > 0
           nCount = nCount+1;
```

```
            %build Reverse Look up table for Plane Route
            aafTruckRouteReverseLookup(nCount,:) = [irow, icol];
            %build Look up table for Plane Route
            aafTruckRouteLookup(irow,icol) = nCount+cDV ;
        end
    end
end

% Remove unnecessary rows from Reverse Look up table
aafTruckRouteReverseLookup = aafTruckRouteReverseLookup(1:nCount,:);
cDV=cDV+nCount;

end

function[aafPlaneRouteLookup,aafPlaneRouteReverseLookup,cDV]=...
    buildXDVPlaneLookup(aafPlaneRoute,cDV)
nSize = size(aafPlaneRoute,1);
%Pre-allocate Variable
    % Plane look up table
    aafPlaneRouteLookup = zeros(nSize);
    % Plane reverse lookup
    aafPlaneRouteReverseLookup = zeros(nSize*nSize,2);

nCount=0;
for irow = 1:nSize
    for icol = 1:nSize
        if aafPlaneRoute(irow,icol) > 0
            nCount = nCount+1;
            %build Reverse Look up table for Plane Route
            aafPlaneRouteReverseLookup(nCount,:) = [irow, icol];
            %build Look up table for Plane Route
            aafPlaneRouteLookup(irow,icol) = nCount ;
        end
    end
end

% Remove unnecessary rows from Reverse Look up table
aafPlaneRouteReverseLookup = aafPlaneRouteReverseLookup(1:nCount,:);
cDV=cDV+nCount;

end

function[acName,afUnitSupply,afPorts,afUnitDemand,...
    aafFlightDistance,aafTruckDistance,...
    aafPlaneRoute,aafTruckRoute,fPlaneCost,fTruckCost,...
    fPlaneSpeed,fTruckSpeed]=getInfo()
%variable n stores number of distribution centers
n = 10;
```

```
%Stores the name of all Distribution center
acName = ["Vancouver","Winnipeg","Toronto","San Francisco","Chicago",...
        "New York", "Denver", "Los Angeles", "Dallas", "Atlanta"];

%number of units supplied from each airport
afUnitSupply = [4000,0,0,0,0,13025,0,6000,0,0];

%Information of Ports wherethe company shipped to
afPorts = [1,6,8];
%Information about units demanded in each distribution center
afUnitDemand = [675,750,2930,885,2715,8625,620,4000,1340,485];

%array that store Dsitance Information between distribution center
    %this array stores the flight distance of all distribution center
    aafFlightDistance =[0,1865,3345,1270,2830,3930,1780,1730,2820,3610;
                        1865,0,1500,2415,1130,2085,1285,2460,1890,2075;
                        3345,1500,0,3642,700,590,2140,3475,1930,1180;
                        1270,2415,3642,0,2965,4150,1525,560,2360,3440;
                        2830,1130,700,2965,0,1120,1455,2785,1290,965;
                        3930,2085,590,4150,1120,0,2640,3955,2240,1210;
                        1780,1285,2140,1525,1455,2640,0,1335,1040,1950;
                        1730,2460,3475,560,2785,3955,1335,0,1970,3110;
                        2820,1890,1930,2360,1290,2240,1040,1970,0,1180;
                        3610,2075,1180,3440,965,1210,1950,3110,1180,0;];
    %this array stores the ground distance of all distribution center
    aafTruckDistance =[0,2303,4357,1538,3475,3002,2323,2058,3526,4465;...
                       2303,0,2031,3061,1371,2711,1762,3153,2069,2566;...
                       4357,2031,0,4236,847,792,2398,4063,2218,1528;...
                       1538,3061,4236,0,3437,4727,2038,623,2754,3980;...
                       3475,1371,847,3437,0,1348,1597,3261,1491,1198;...
                       3002,2711,792,4727,1348,0,2887,4546,2554,1440;...
                       2323,1762,2398,2038,1597,2887,0,1700,1211,2250;...
                       2058,3153,4063,623,3261,4546,1700,0,2325,3546;...
                       3526,2069,2218,2754,1491,2554,1211,2325,0,1299;...
                       4465,2566,1528,3980,1198,1440,2250,3546,1299,0];

%nxn array that stores all the route available
aafPlaneRoute = buildPlaneRoute(n,acName);
aafTruckRoute = buildTruckRoute(n,acName);

%Variable that store the cost from different routes
fPlaneCost = 5.79; %$5.79 /(km*unit)
fTruckCost = 2.76;  %$2.1 /(km*unit)

%Variable that store Information about speed
fPlaneSpeed = 835; % km/hour
fTruckSpeed = 79;  %km/hour
end
```

```
function[A]=buildTruckRoute(n,acName)
%This Function build adjacency matrix that stores distribution centers

% Initialize Adjacency matrix
A = zeros(n);
%Filling in active node
A(1,2)  = 3000; %Capacity from 1 to 2 is 3000
A(1,4)  = 2000; %Capacity from 1 to 4 is 2000
A(2,5)  = 1500; %Capacity from 2 to 5 is 1500
A(2,7)  = 4000; %Capacity from 2 to 7 is 4000
A(3,2)  = 5000; %Capacity from 3 to 2 is 5000
A(5,6)  = 4000; %Capacity from 5 to 6 is 4000
A(5,10) = 2000; %Capacity from 5 to 10 is 4000
A(6,3)  = 3000; %Capacity from 6 to 3 is 3000
A(6,10) = 3000; %Capacity from 6 to 10 is 3000
A(7,5)  = 3000; %Capacity from 7 to 5 is 3000
A(8,4)  = 500;  %Capacity from 8 to 4 is 500
A(8,7)  = 3000; %Capacity from 8 to 7 is 3000
A(9,8)  = 500;  %Capacity from 9 to 8 is 500
A(10,9) = 2000; %Capacity from 10 to 9 is 2000

 A=1.5*A;
% for irow = 1:n
%     for icol = 1:n
%         if A(irow,icol) >0
%         fprintf('Capacity from %s to %s is %.3f\n',...
%             acName(irow),acName(icol),A(irow,icol))
%         end
%     end
% end

end

function[A]=buildPlaneRoute(n,acName)
%This Function build adjacency matrix that stores distribution centers

% Initialize Adjacency matrix
A = zeros(n);
%Filling in active node
A(1,2)  = 3000; %Capacity from 1 to 2 is 3000
A(1,4)  = 2000; %Capacity from 1 to 4 is 2000
A(2,5)  = 1500; %Capacity from 2 to 5 is 1500
A(2,7)  = 4000; %Capacity from 2 to 7 is 4000
A(3,2)  = 5000; %Capacity from 3 to 2 is 5000
A(5,6)  = 4000; %Capacity from 5 to 6 is 4000
A(5,10) = 2000; %Capacity from 5 to 10 is 4000
A(6,3)  = 3000; %Capacity from 6 to 3 is 3000
```

```
A(6,10) = 3000; %Capacity from 6 to 10 is 3000
A(7,5)  = 3000; %Capacity from 7 to 5 is 3000
A(8,4)  = 500;  %Capacity from 8 to 4 is 500
A(8,7)  = 3000; %Capacity from 8 to 7 is 3000
A(9,8)  = 500;  %Capacity from 9 to 8 is 500
A(10,9) = 2000; %Capacity from 10 to 9 is 2000

% for irow = 1:n
%     for icol = 1:n
%         if A(irow,icol) >0
%         fprintf('Capacity from %s to %s is %.3f\n',...
%             acName(irow),acName(icol),A(irow,icol))
%         end
%     end
% end

end
```